

31338 Network Servers

Lab 8b: Samba

Aims

1. To configure a Linux machine to act as a Samba server and create a Samba share
2. Access the Samba share from a Windows or Linux machine

Task 1: Samba basics

To install Samba and create a Samba share

1. On the Linux virtual machine, ensure the ens37 interface uses a static network address, e.g. 10.0.2.1/24.
2. Verify the Samba package is installed. Use `dnf` to install the samba package if `rpm` says it is not on the machine. `rpm -q samba`
3. Make a backup of the file `/etc/samba/smb.conf` as `/etc/samba/smb.conf_backup`. You can recover the `smb.conf` later in case you need to.

Now have a look at the `smb.conf` file. The lines that you particularly need to pay attention to are shown below. Most are quite well documented in the example file in `/etc/samba`. Make your changes to the file.

- `workgroup` (e.g. set to “WORKGROUP”, which is the default for Windows machines)
 - `netbios name` (set to “MYSAMBASERVER” – this becomes the machine's "Samba hostname")
 - `interfaces` (e.g. use “10.0.2.0/24” and “127.0.0.0/8”)
 - `hosts allow` (set to “10.0.2.” – note the unusual usage of dots)
4. Now check/edit the different directories you will share with Samba. For now, stick with `[homes]`, `[printers]` and `[print$]` (we don’t have any printers, so the last two won’t actually do anything useful – you could also comment them out if you like). The `[homes]` section is the default share of home directories.
 5. Under the `[homes]` section, set the *browseable* option to **Yes** and *read only* option to **Yes**.
 6. To verify that your changes look okay, use the `testparm` utility:

```
testparm smb.conf
```

7. To allow Samba traffic, sometimes the firewall and SELinux currently protecting the Linux system need to be configured. In our case SELinux is disabled, but we should configure the firewall. To do that, first run:

```
firewall-config
```

Check which zone your ens37 interface is in (probably the Default Zone – public). In that case, click on the public zone on the right-hand side, and scroll the list of services to find samba. Turn on the checkbox beside samba. Near the top of the interface you should see an option that says “Configuration: runtime”. That means that we only changed the firewall for the current runtime, not permanently. Change the drop-down to say “Configuration: permanent” and make the same change (turn on samba in the public zone).

Note: If this was a production system, we might have changed the ens37 interface to a “trusted” firewall zone, and kept ens33 in the “public” zone, reflecting the two different levels of security on the networks we are using. If you want to use that setup now, you can do that here too, but be aware that if you have other services you are sharing from Linux to Windows you would need to also add them into the trusted zone (and possibly remove them from the public zone).

8. Any user that requires access to a Samba shared resource must be configured as a Samba user and assigned a password. Use the `pdbedit` command to set up Samba accounts for your existing Unix users, e.g.

```
pdbedit -a root
pdbedit -a peter
etc.
```

```
pdbedit -L          will show you a list of the current users
pdbedit -L -v       will show you a more verbose listing
pdbedit -x          will let you delete a user from the Samba password database if you need to
```

9. Start up the `smb` and `nmb` services with `systemctl`. Also use `systemctl` to enable the services so they start when the system boots up.

Task 2: Testing Samba from Linux

First test Samba from your Linux virtual machine. View a list of your shares with:

```
smbclient -L 10.0.2.1
```

You will be prompted for a password. Try it without any password (just hit Enter), and you see a list of the shares that are publicly (anonymously) visible. Try it with the Samba root password and you see a list of shares available to the root user. What is different between the two lists? (Note: you must have used `pdbedit` earlier to set up a Samba password for root, otherwise you will see an `NT_STATUS_LOGON_FAILURE` error).

Now try connecting to the home directory for one of your users (e.g. peter). You will have to choose a user that you gave a Samba password to earlier.

```
smbclient -U peter //10.0.2.1/peter
```

`smbclient` provides an interface vaguely similar to command-line FTP. Verify that you can see the contents of the user's home directory through `smbclient` (`dir` command). While you are "in" `smbclient`, you can type `?` to see a list of `smbclient` commands.

Task 3: Testing Samba from Windows Server

Next, try to connect to the home directory from your Windows Server virtual machine. Open Windows File Explorer (note: File Explorer, not Internet Explorer), and in the address bar, type in **\\10.0.2.1\peter**

If you get an error that it cannot connect, troubleshoot by first checking that you can ping the server. If you can ping, then it could be that on the Linux server, the firewall is blocking incoming samba connections. See Task 1 for configuring the Linux firewall (maybe you didn't make your firewall changes permanent?)

Once you are connected to the share, try to create a file or directory there. If you get an "Access is denied" message, make a change in `smb.conf` on the Linux server so users have write access. Use `systemctl` to restart the `smb` service.

Task 4: Creating your own shares on Linux

Once you have Samba working, try configuring the following:

1. Share the `/tmp` directory of your Linux server to Windows clients. The tmp share should be browseable, writeable and public, but all files created by Windows users in `/tmp` should be owned by the UNIX user “nobody” (not the actual user who is logged in). Read about the Samba “force user” option, e.g.

```
force user = nobody
```
2. Share the `/opt` directory of your Linux server to Windows clients. The opt share should be read-only, but publicly available and browseable.

Remember to use `testparm` to check your configuration. It may alert you if your configuration isn't valid.

Test that you can access both of these shares on Windows. In Windows, try browsing just: \\10.0.2.1
Which shares do you see? Document all of this in your journal.

Task 5: Creating your own shares on Windows Server

Creating SMB shares in Windows is much simpler, because SMB is the native Windows approach to file sharing. Also notice that with Windows we don't call it “Samba”. Samba is the name of a Linux software package that implements a Linux SMB server. On Windows, it's just file sharing.

1. Open File Explorer on Windows (File Explorer, not Internet Explorer).
2. Create a folder **c:\winshare**. Inside this folder create a file, e.g. “mywinfile.txt”.
3. Right-click on this folder and choose Give access to  Specific people
4. On the window that appears, type “stewie” as the username of the person to share with (it must be a username that you previously created on the Windows Server). You can choose either “Read/write” or “Read” from the drop-down.
5. Importantly, click the “Share” button at the bottom to save these changes. You should see a confirmation that the folder is shared.
6. On Linux, now run the command:

```
smbclient -U stewie //10.0.2.2/winshare
```

Notes: with the `-U` option, make sure it is an uppercase ‘U’. Also note that the IP address is for the Windows Server (change it if your Windows Server has a different IP address on Ethernet1). Also note that in Linux, we can use forward slashes instead of backslashes (backslashes have special meaning to Unix shells).

Also note: for the password, you need to enter stewie's password on the **Windows Server** not his Linux password (just in case you had set different passwords on Linux and Windows).

7. In Linux, you should now be able to see the file you created on the Windows Server. Try the following commands inside `smbclient`, and make notes in your journal. Verify that the files you uploaded/downloaded are where you expected them to be.

```
dir
get mywinfile.txt /tmp/win
put /etc/samba/smb.conf samba.txt
```